Binary Trees Lab

Data Structures & Algorithms

Lab 05 -- Your First Tree: Building and Exploring Binary Trees

Duration: 2 hours (2 challenges)

Lab Structure

Each challenge follows this format:

1. **Challenge Presentation** (5 min)
2. **Your Implementation Time** (25 min)
3. **Pseudocode Review** (5 min)
4. **Solution Walkthrough** (10 min)
5. **Code Explanation** (15 min)

Total per challenge: ~60 minutes

Today's Challenges

Challenge 1: Build Your First Binary Tree

- Create tree nodes, connect them with pointers
- Walk through the tree using **in-order** and **level-order** traversals
- Understand the basic building blocks: `struct Node`, `createNode`, and recursive traversal

Challenge 2: Trees vs. Arrays -- The Department Directory

- Model a **company hierarchy** as a binary tree
- Try the same task with a **flat array** and see how awkward it gets
- Answer real questions about the hierarchy: "How many people are under the CTO?", "Who are the managers?"
- See firsthand **why trees exist** -- some data simply does not fit in a line

Challenge 1

Build Your First Binary Tree

Challenge 1: Problem Statement

Task: Build a binary tree by creating individual nodes and linking them with pointers. Then walk through the tree using two traversal methods.

The tree to build:

```
          1
         / \
        2   3
       / \
      4   5
```

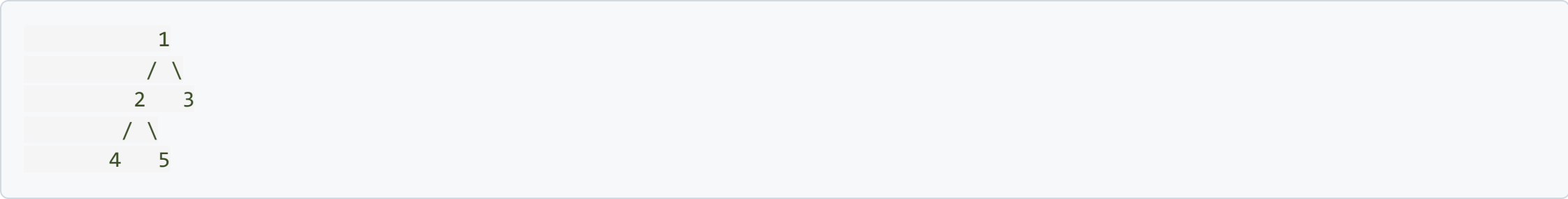
Requirements:

- Define a `Node` struct with `data`, `left`, and `right`
- Write `createNode(value)` to allocate and initialize a new node
- Manually link nodes to build the tree above
- Write **in-order** traversal (Left, Node, Right) using recursion
- Write **level-order** traversal (level by level) using a simple array-based queue
- Print both traversal outputs

Time: 25 minutes

Challenge 1: Think About It First

Before coding, trace through the tree by hand:



In-order (Left, Node, Right):

Start at 1 -> go left to 2 -> go left to 4 -> no left child -> print **4** -> back to 2 -> print **2** -> go right to 5 -> print **5** -> back to 1 -> print **1** -> go right to 3 -> print **3**

Result: `4 2 5 1 3`

Level-order (top to bottom, left to right):

Level 0: **1** -> Level 1: **2, 3** -> Level 2: **4, 5**

Result: `1 2 3 4 5`

Challenge 1: Pseudocode

```
ALGORITHM BuildAndTraverseTree

  1. Define Node structure:
     - int data
     - pointer to left child (initially NULL)
     - pointer to right child (initially NULL)

  2. createNode(value):
     - Allocate memory with malloc
     - Set data = value, left = NULL, right = NULL
     - Return the new node

  3. Build tree:
     - root = createNode(1)
     - root->left = createNode(2),  root->right = createNode(3)
     - root->left->left = createNode(4)
     - root->left->right = createNode(5)

  4. inorder(node):
     - If node is NULL: return (base case)
     - inorder(node->left)      -- go left first
     - Print node->data         -- then visit this node
     - inorder(node->right)     -- then go right

  5. levelorder(root):
     - Put root into a queue
     - While queue is not empty:
         - Take the front node out
         - Print its data
         - Put its left child in the queue (if it exists)
         - Put its right child in the queue (if it exists)

END ALGORITHM
```

Challenge 1: Solution Code -- Node and Creation

```c
#include <stdio.h>
#include <stdlib.h>

// The building block of a binary tree
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create a new node and initialize it
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data  = value;
    newNode->left  = NULL;
    newNode->right = NULL;
    return newNode;
}
```

This is a **self-referential structure** -- just like the nodes we built for linked lists, but with **two** pointers instead of one. That single change transforms a chain into a tree.

Challenge 1: Solution Code -- In-Order Traversal

```c
// In-order traversal: Left -> Node -> Right
void inorder(struct Node* node) {
    if (node == NULL) return;   // Base case: nothing to visit

    inorder(node->left);        // 1. Visit everything on the left
    printf("%d ", node->data);  // 2. Visit this node
    inorder(node->right);       // 3. Visit everything on the right
}
```

That is the entire function -- just 4 lines of real code.

Recursion handles all the complexity. The call stack remembers where to return after visiting subtrees. We do not need any loops, counters, or extra data structures.

Why "In-order"? Because the node is visited **in between** its left and right subtrees. If this were a BST (Binary Search Tree), in-order traversal would print the values in sorted order -- a useful property we will explore in later lectures.

Challenge 1: Solution Code -- Level-Order Traversal

```c
// Level-order traversal: visit nodes level by level (uses a queue)
void levelorder(struct Node* root) {
    if (root == NULL) return;

    // Simple array-based queue (big enough for our small tree)
    struct Node* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;                // Start by putting the root in the queue

    while (front < rear) {
        struct Node* current = queue[front++];   // Take one node out
        printf("%d ", current->data);            // Visit it

        // Put its children in the queue (they will be visited next level)
        if (current->left  != NULL) queue[rear++] = current->left;
        if (current->right != NULL) queue[rear++] = current->right;
    }
}
```

Why a queue? A queue processes items in the order they were added (FIFO). By adding all children of the current level before moving on, we guarantee level-by-level processing. This is exactly the **BFS** (Breadth-First Search) pattern from our Queues unit.

Challenge 1: Solution Code -- Main Program

```
int main() {
    printf("=== Challenge 1: Build Your First Binary Tree ===\n\n");

    // Step 1: Create individual nodes
    struct Node* root = createNode(1);
    struct Node* n2   = createNode(2);
    struct Node* n3   = createNode(3);
    struct Node* n4   = createNode(4);
    struct Node* n5   = createNode(5);

    // Step 2: Wire them together
    root->left  = n2;        //     1
    root->right = n3;        //    / \
    n2->left    = n4;        //   2   3
    n2->right   = n5;        //  / \
                             // 4   5
    // Step 3: Traverse the tree
    printf("In-order    (L, N, R): ");
    inorder(root);
    printf("\n");

    printf("Level-order (BFS)   : ");
    levelorder(root);
    printf("\n");

    return 0;
}
```

Challenge 1: Expected Output

```
=== Challenge 1: Build Your First Binary Tree ===

In-order    (L, N, R): 4 2 5 1 3
Level-order (BFS)    : 1 2 3 4 5
```

What just happened?

- We created 5 separate nodes in memory using `malloc`
- We connected them by setting `left` and `right` pointers
- We walked through ALL of them using just 4 lines of recursive code
- We also walked through them level-by-level using a queue

Compare this with a linked list: A linked list can only go forward. A tree can branch. That is the fundamental difference -- and it enables everything we will build in the upcoming lectures.

Challenge 1: Code Explanation

Part 1: Node Creation -- What Happens in Memory

```
struct Node* root = createNode(1);
```

When this line runs, `malloc` carves out a chunk of memory on the **heap**:

```
Memory (heap):
  +------+------+-------+
  | data | left | right |
  |  1   | NULL | NULL  |
  +------+------+-------+
     ^
     |
   root (pointer on the stack)
```

After wiring `root->left = n2` and `root->right = n3`:

```
  root -> [1 | * | * ]
               |   |
               v   v
           [2|*|*] [3|N|N]
            |  |
            v  v
        [4|N|N] [5|N|N]
```

Each arrow is a pointer. Each `N` is a NULL pointer (no child).

Challenge 1: Code Explanation

Part 2: How Recursion Walks the Tree

Let us trace `inorder(root)` step by step:

```
inorder(1):
  --> inorder(2):
        --> inorder(4):
              --> inorder(NULL): return          (no left child)
              PRINT 4                             << first output
              --> inorder(NULL): return          (no right child)
        PRINT 2                                   << second output
        --> inorder(5):
              --> inorder(NULL): return
              PRINT 5                             << third output
              --> inorder(NULL): return
  PRINT 1                                         << fourth output
  --> inorder(3):
        --> inorder(NULL): return
        PRINT 3                                   << fifth output
        --> inorder(NULL): return
```

Output: 4 2 5 1 3 -- The recursion dives as deep left as possible, then unwinds, visiting each node "in order".

Challenge 1: Code Explanation

Part 3: Level-Order -- Watching the Queue Work

```
Start:          Queue: [1]              Output: (empty)

Step 1: Dequeue 1.  Print 1.
        Enqueue children: 2 and 3.
        Queue: [2, 3]                   Output: 1

Step 2: Dequeue 2.  Print 2.
        Enqueue children: 4 and 5.
        Queue: [3, 4, 5]               Output: 1 2

Step 3: Dequeue 3.  Print 3.
        No children to enqueue.
        Queue: [4, 5]                   Output: 1 2 3

Step 4: Dequeue 4.  Print 4.
        No children.
        Queue: [5]                      Output: 1 2 3 4

Step 5: Dequeue 5.  Print 5.
        No children.
        Queue: []                       Output: 1 2 3 4 5
```

The queue ensures nodes at the same level are processed together. By the time we start level 2, all level 1 nodes are already done.

Challenge 2

Trees vs. Arrays -- The Department Directory

Challenge 2: The Scenario

You are building an employee directory for a small company. Here is the hierarchy:

```
                  CEO (Alice)
                  /          \
          CTO (Bob)          CFO (Carol)
          /       \                \
    Dev Lead     QA Lead      Accountant
    (Dave)       (Eve)        (Frank)
```

Your task: Answer these questions about the company:

1. Print everyone in the company
2. How many employees are there in total?
3. How many people report under the CTO (directly or indirectly)?
4. Who are the "leaf" employees (people with no one reporting to them)?

The twist: You will solve this **twice** -- first with a **flat array**, then with a **tree**. See which approach feels more natural, more readable, and more maintainable.

Time: 25 minutes

Challenge 2: Attempt 1 -- The Array Approach

How would you store this hierarchy in a flat array?

You would need extra information to track who reports to whom:

```
// Flat array approach -- storing employees and their relationships
char* employees[] = {"Alice", "Bob", "Carol", "Dave", "Eve", "Frank"};
int   parentIndex[] = {  -1,     0,      0,      1,     1,      2   };
//                      CEO   reports  reports  reports reports reports
//                      (no    to       to       to     to      to
//                     parent) Alice    Alice     Bob    Bob    Carol
```

Problems already appearing:

- The hierarchy is not visible from looking at the data
- To find "everyone under Bob", we must **scan the entire array** checking `parentIndex`
- Adding a new level means rethinking the indexing
- Every operation requires **linear search**: O(n) to find anything

Challenge 2: Array Solution Code

```c
#include <stdio.h>
#include <string.h>

#define NUM_EMPLOYEES 6

char* employees[] = {"Alice", "Bob", "Carol", "Dave", "Eve", "Frank"};
int parentIndex[] = {-1, 0, 0, 1, 1, 2};
//                   CEO reports-to-Alice  reports-to-Bob  reports-to-Carol

// Print everyone (trivial with an array)
void printAll_Array() {
    printf("All employees: ");
    for (int i = 0; i < NUM_EMPLOYEES; i++)
        printf("%s  ", employees[i]);
    printf("\n");
}
```

So far so good. But now watch what happens when we try to answer hierarchical questions...

Challenge 2: Array Solution -- Hierarchical Queries

```
// Count people under a given employee (including indirect reports)
int countUnder_Array(int bossIndex) {
    int count = 0;
    for (int i = 0; i < NUM_EMPLOYEES; i++) {
        if (parentIndex[i] == bossIndex) {
            count++;                              // direct report
            count += countUnder_Array(i);         // indirect reports
        }
    }
    return count;
}

// Find leaf employees (no one reports to them)
void findLeaves_Array() {
    printf("Leaf employees (no reports): ");
    for (int i = 0; i < NUM_EMPLOYEES; i++) {
        int hasReport = 0;
        for (int j = 0; j < NUM_EMPLOYEES; j++) {   // scan ENTIRE array
            if (parentIndex[j] == i) { hasReport = 1; break; }
        }
        if (!hasReport) printf("%s  ", employees[i]);
    }
    printf("\n");
}
```

Finding leaves requires a **nested loop** -- O(n^2). Every query scans the full array.

Challenge 2: Array Solution -- Main Program

```c
int main() {
    printf("=== ARRAY APPROACH ===\n\n");

    printAll_Array();

    printf("Total employees: %d\n", NUM_EMPLOYEES);

    // Find Bob's index
    int bobIndex = -1;
    for (int i = 0; i < NUM_EMPLOYEES; i++) {
        if (strcmp(employees[i], "Bob") == 0) { bobIndex = i; break; }
    }
    printf("People under Bob: %d\n", countUnder_Array(bobIndex));

    findLeaves_Array();

    return 0;
}
```

Output:

```
All employees: Alice  Bob  Carol  Dave  Eve  Frank
Total employees: 6
People under Bob: 2
Leaf employees (no reports): Dave  Eve  Frank
```

It works. But the code is clunky, and every question requires scanning the whole array.

Challenge 2: Now Do It With a Tree

Same company, but stored as a tree:

```
struct Employee {
    char name[50];
    struct Employee* left;    // first subordinate
    struct Employee* right;   // second subordinate
};

struct Employee* createEmployee(const char* name) {
    struct Employee* emp = (struct Employee*)malloc(sizeof(struct Employee));
    if (emp == NULL) { printf("Memory error!\n"); exit(1); }
    strcpy(emp->name, name);
    emp->left  = NULL;
    emp->right = NULL;
    return emp;
}
```

The structure naturally represents the hierarchy. No extra `parentIndex` array needed.

Challenge 2: Tree Solution -- Building the Hierarchy

```
struct Employee* buildCompany() {
    struct Employee* ceo = createEmployee("Alice");

    // Alice's direct reports
    ceo->left  = createEmployee("Bob");
    ceo->right = createEmployee("Carol");

    // Bob's team
    ceo->left->left  = createEmployee("Dave");
    ceo->left->right = createEmployee("Eve");

    // Carol's team
    ceo->right->right = createEmployee("Frank");

    return ceo;
}
```

Notice: The code itself mirrors the org chart. You can *see* the hierarchy in the code.

Challenge 2: Tree Solution -- Answering Questions

```c
// Print everyone (in-order traversal)
void printAll_Tree(struct Employee* emp) {
    if (emp == NULL) return;
    printAll_Tree(emp->left);
    printf("%s  ", emp->name);
    printAll_Tree(emp->right);
}

// Count all employees in a subtree
int countAll_Tree(struct Employee* emp) {
    if (emp == NULL) return 0;
    return 1 + countAll_Tree(emp->left) + countAll_Tree(emp->right);
}

// Count people UNDER a given employee (not including them)
int countUnder_Tree(struct Employee* boss) {
    if (boss == NULL) return 0;
    return countAll_Tree(boss) - 1;   // subtract the boss themselves
}

// Find leaf employees (no subordinates)
void findLeaves_Tree(struct Employee* emp) {
    if (emp == NULL) return;
    if (emp->left == NULL && emp->right == NULL)
        printf("%s  ", emp->name);
    findLeaves_Tree(emp->left);
    findLeaves_Tree(emp->right);
}
```

Challenge 2: Tree Solution -- Main Program

```c
int main() {
    printf("=== TREE APPROACH ===\n\n");

    struct Employee* ceo = buildCompany();

    printf("All employees: ");
    printAll_Tree(ceo);
    printf("\n");

    printf("Total employees: %d\n", countAll_Tree(ceo));

    // Find people under Bob -- we have a DIRECT pointer to Bob's subtree!
    struct Employee* bob = ceo->left;
    printf("People under Bob: %d\n", countUnder_Tree(bob));

    printf("Leaf employees: ");
    findLeaves_Tree(ceo);
    printf("\n");

    return 0;
}
```

Output:

```
All employees: Dave  Bob  Eve  Alice  Carol  Frank
Total employees: 6
People under Bob: 2
Leaf employees: Dave  Eve  Frank
```

Challenge 2: Code Explanation

Part 1: The Side-by-Side Comparison

Question	Array Approach	Tree Approach
Print all	Simple loop	Simple recursion
Count total	Use a constant	3-line recursion
Count under boss	Scan entire array, recursion on indices	Just count subtree -- 1 line
Find leaves	**Nested loop** (O(n^2))	Simple recursion (O(n))
Add a new employee	Add to arrays, set parent index	Set a pointer on the parent node
See the hierarchy	Cannot tell from looking at arrays	The code mirrors the org chart

The tree wins on every hierarchical question. This is not because trees are always better -- for flat data (a shopping list, a to-do list), arrays are perfect. But when data has a natural **parent-child structure**, trees are the right tool.

Challenge 2: Code Explanation

Part 2: Why `countUnder` is Simpler with a Tree

Array version: Must scan the entire array to find children, then recurse on each child.

```
int countUnder_Array(int bossIndex) {
    int count = 0;
    for (int i = 0; i < NUM_EMPLOYEES; i++) {     // scan ALL employees
        if (parentIndex[i] == bossIndex) {
            count++;
            count += countUnder_Array(i);
        }
    }
    return count;
}
```

Tree version: Just count the subtree. Children are directly accessible via pointers.

```
int countAll_Tree(struct Employee* emp) {
    if (emp == NULL) return 0;
    return 1 + countAll_Tree(emp->left) + countAll_Tree(emp->right);
}
```

The tree version never looks at nodes outside the relevant subtree. If Bob manages 2 people in a company of 10,000, the tree version only visits 3 nodes. The array version still scans all 10,000.

Challenge 2: Code Explanation

Part 3: Why Leaf Detection Shows the Biggest Difference

Array -- finding leaves requires O(n^2):

```
for (int i = 0; i < n; i++) {              // for each employee
    int hasReport = 0;
    for (int j = 0; j < n; j++) {          // check if anyone reports to them
        if (parentIndex[j] == i) { hasReport = 1; break; }
    }
    if (!hasReport) printf("%s ", employees[i]);
}
```

Tree -- finding leaves requires O(n):

```
void findLeaves(struct Employee* emp) {
    if (emp == NULL) return;
    if (emp->left == NULL && emp->right == NULL)   // just check own pointers
        printf("%s ", emp->name);
    findLeaves(emp->left);
    findLeaves(emp->right);
}
```

In the tree, each node already **knows** whether it has children -- just check if its pointers are NULL. No searching required! In the array, you have to scan the *entire* array to determine if anyone reports to you.

Challenge 2: Code Explanation

Part 4: The Key Takeaway

```
The right data structure makes the problem easy.
The wrong data structure makes the problem hard.
```

Data Shape	Best Structure	Why
Flat list (groceries, to-do)	Array or Linked List	No hierarchy, just sequence
LIFO operations (undo, back)	Stack	Need last-in-first-out
FIFO operations (queue, buffer)	Queue	Need first-in-first-out
Hierarchy (org chart, files)	**Tree**	**Parent-child relationships**

We chose to study trees not because they are fancier, but because **real-world problems demand them**. File systems, HTML documents, database indexes, compiler parse trees, decision trees in AI -- all are hierarchical.

In the upcoming lectures, we will add **ordering rules** (BSTs) and **balancing** (AVL trees) to make trees even more powerful.

Lab Session Complete!

Summary: What You Built

Challenge 1: Build Your First Binary Tree

- Created tree nodes using `struct` and `malloc` -- the same self-referential pattern from linked lists, but with two pointers
- Linked nodes together to form a tree
- Traversed the tree with recursion (in-order) and with a queue (level-order)
- Saw how 4 lines of recursive code can walk through an entire tree

Challenge 2: Trees vs. Arrays -- The Department Directory

- Stored the same hierarchy in two ways: a flat array and a tree
- Discovered that hierarchical questions (count subtree, find leaves) are **natural and efficient** with trees but **awkward and slow** with arrays
- Understood that trees are not a replacement for arrays -- they solve a *different kind of problem*

Key Concepts Reinforced

Self-Referential Structures

```
struct Node {                    struct Node {
    int data;                        int data;
    struct Node* next;    -->        struct Node* left;
};                                   struct Node* right;
// Linked List node              };
//   (one direction)              // Tree node (two directions)
```

One pointer makes a chain. Two pointers make a tree. That is the only structural change -- but it opens up an entirely new world of data organization.

Recursion is the Natural Language of Trees

- **Base case:** `if (node == NULL) return`
- **Recursive case:** process left, process self, process right
- The call stack handles all the bookkeeping

Queues Revisited

- Level-order traversal is BFS -- the same queue pattern from our Queues unit

Complexity Summary

Operation	Array Approach	Tree Approach
Print all employees	O(n)	O(n)
Count total	O(1) with constant	O(n)
Count under a boss	O(n) per level (scan)	O(k) where k = subtree size
Find leaves	**O(n^2)** (nested scan)	**O(n)** (single pass)
See hierarchy in code	Not possible	Natural
Add new employee under X	O(1) to append + update index	O(1) pointer update

Trees shine when the data is hierarchical and when queries involve **subtrees** -- "everything under this point." Arrays shine when the data is flat and you need random access by index.

Take-Home Challenge (Optional)

Extend your department directory with these features:

1. **Search by Name:** Write a function `findEmployee(root, name)` that searches the tree for an employee by name and returns a pointer to their node (or NULL if not found). Print a message like "Found Dave -- they report under Bob."

2. **Team Printer:** Write a function `printTeam(boss)` that prints a boss and all the people under them with indentation to show the hierarchy:

```
Alice
  Bob
    Dave
    Eve
  Carol
    Frank
```

   Hint: Pass a "depth" parameter and print spaces based on depth.

3. **Think about it:** What happens if a manager has **three** direct reports, not just two? Can a binary tree handle this? What would you need to change? (We will study general trees and multi-way trees later in this unit.)